Guía de estudio y de trabajo

# Del prototipo a producción

Todo lo que debes saber, investigar y hacer para pasar el sistema de la heladería, que hoy es un archivo HTML con datos en el navegador, a un sistema real con base de datos, inicio de sesión y buenas prácticas, empezando con costo cero.

---

Sistema de heladería artesanal · Administración y colaboradores
Noviembre de 2026
Los precios y límites de los servicios cambian: verifícalos en sus páginas oficiales antes de decidir.

# Contenido


1. El panorama en una página
2. Conceptos básicos que debes dominar
3. La arquitectura elegida y por qué
4. Herramientas: qué es cada una y para qué la usaremos
5. Organización del proyecto
6. Git y forma de trabajo
7. Base de datos: diseño y buenas prácticas
8. Inicio de sesión y permisos
9. Funcionar sin internet
10. Calidad del código y pruebas
11. Integración y despliegue continuo
12. Los tres entornos y el paso a producción
13. Seguridad
14. Operación: respaldos y monitoreo
15. Costos y límites de los planes gratis
16. Lista de verificación antes de producción
17. Plan de estudio sugerido
18. Glosario de la A a la Z


**Cómo usar esta guía.** Léela una vez de principio a fin para tener el mapa completo. Después usa el capítulo 17 (plan de estudio) como orden de trabajo y el 18 (glosario) como diccionario. Cada término técnico aparece explicado en un recuadro azul la primera vez que se usa.

# 1. El panorama en una página

Hoy el sistema es **un solo archivo** (`index.html`) publicado en GitHub Pages. Guarda todo en el navegador de cada celular. Eso tiene tres problemas para usarlo en serio:

- Lo que anota una colaboradora no lo ve la administradora en su celular.
- Si se borra el navegador o se pierde el celular, se pierden los datos.
- El inicio de sesión es de mentira: cualquiera con conocimientos puede saltárselo.

La meta es tener **una sola base de datos en internet**. Todos los celulares leen y escriben en ella, cada persona entra con su cuenta y la base de datos decide qué puede ver y hacer cada una.

## El recorrido completo

| 1. Tu PC | 2. Pruebas | 3. Ensayo | 4. Producción | 5. Operación |
|---|---|---|---|---|
| Desarrollo local con base de datos en Docker | Automáticas en GitHub en cada cambio | Entorno de prueba en internet, con celulares reales | El sistema que usa la heladería | Respaldos, monitoreo y mejoras |

## Las piezas, en resumen

| Pieza | Herramienta | Costo inicial |
|---|---|---|
| Código y su historial | Git + GitHub (repositorio privado) | Gratis |
| Lenguaje | TypeScript | Gratis |
| Armar la app | Vite | Gratis |
| Base de datos, login y permisos | Supabase (PostgreSQL) | Plan Free |
| Base de datos en tu PC | Docker Desktop + Supabase CLI | Gratis |
| Funcionar sin señal | PWA: vite-plugin-pwa + IndexedDB (Dexie.js) | Gratis |
| Pruebas | Vitest, Playwright, pgTAP | Gratis |
| Automatización | GitHub Actions | Gratis (2.000 min/mes) |
| Publicar la app | Cloudflare Pages | Gratis |
| Avisos de errores y caídas | Sentry + UptimeRobot | Gratis |

> **Lo que pagarás después.** Cuando el negocio lo justifique: Supabase Pro (unos US$ 25/mes) para tener respaldos automáticos y evitar que la base de datos se pause. Opcional: un dominio propio (unos US$ 10–11/año).

## 2. Conceptos básicos que debes dominar

Antes de las herramientas, estos conceptos. Si los entiendes, todo lo demás encaja.

**Cliente y servidor.** El **cliente** es el programa que usa la persona: la página en el navegador del celular. El **servidor** es una computadora en internet que guarda los datos y responde pedidos. El cliente pide ("dame las ventas de hoy") y el servidor responde.

**Frontend y backend.** **Frontend** es todo lo que se ve y corre en el celular: pantallas, botones, estilos. **Backend** es lo que corre en el servidor: base de datos, reglas, login. En este proyecto el backend lo da Supabase y tú programas sobre todo el frontend.

**Base de datos relacional y SQL.** Una base de datos **relacional** guarda la información en **tablas** (filas y columnas), como hojas de Excel conectadas entre sí. Por ejemplo, cada *pedido* apunta a la *caja* donde se cobró. **SQL** es el lenguaje para crear tablas y consultar datos: `SELECT * FROM pedidos WHERE fecha = '2026-11-02'`. **PostgreSQL** (o "Postgres") es la base de datos relacional que usaremos. Es gratuita, muy usada y muy confiable.

**API.** Significa *Application Programming Interface* (interfaz de programación). Es la "ventanilla" por la que un programa le pide cosas a otro. Supabase crea automáticamente una API a partir de tus tablas. Desde la app escribes `supabase.from('pedidos').insert(...)` sin programar un servidor.

**HTTP, HTTPS y JSON.** **HTTP** es el idioma con el que el navegador y el servidor se hablan. **HTTPS** es lo mismo pero cifrado: nadie en medio puede leer las contraseñas ni los datos. **JSON** es el formato de texto en que viajan los datos: `{"sabor":"Canela","precio":8}`.

**Entorno (environment).** Una copia completa del sistema que funciona por separado. Tendremos tres. **Desarrollo**: tu PC, donde puedes romper todo. **Pruebas** (staging): en internet, para ensayar con celulares reales. **Producción**: el que usa la heladería con datos reales.

**Despliegue (deploy).** Publicar una versión nueva del sistema en un entorno. "Desplegar a producción" es dejar la versión nueva disponible para la heladería.

**Variable de entorno y secreto.** Un valor de configuración que no se escribe en el código, sino que se le da al programa desde afuera, por ejemplo la dirección de la base de datos. Un **secreto** es una variable de entorno que nadie debe ver, como una contraseña o una llave privada. Los secretos **nunca** se suben a Git.

**Repositorio.** La carpeta del proyecto con todo su historial de cambios, administrada por Git y guardada en GitHub.

**Dependencia y npm.** Una **dependencia** es código de otras personas que tu proyecto usa (por ejemplo, la librería de Supabase). **npm** es el programa que las descarga e instala. La lista queda en `package.json`.

**Node.js.** Programa que permite ejecutar JavaScript fuera del navegador, en tu PC. Lo necesitas para las herramientas de desarrollo (Vite, pruebas, Supabase CLI), aunque la app final corra en el navegador.

# 3. La arquitectura elegida y por qué

> **Arquitectura.** Es la forma en que se organizan las piezas de un sistema y cómo se comunican. Elegirla bien desde el inicio evita reescribir todo después.

La arquitectura de este proyecto combina tres ideas:

## 3.1 Cliente–servidor con BaaS

> **BaaS (Backend as a Service).** "Backend como servicio": en lugar de programar y mantener tu propio servidor, contratas uno ya hecho que te da base de datos, login, permisos y almacenamiento de archivos. Aquí el BaaS es **Supabase**.

Por qué: para 1 administradora y 2 o 3 colaboradoras, un servidor propio (Node.js con Express, o Java con Spring) significa más código, más cosas que actualizar, más puntos de falla y, casi siempre, más costo. Con Supabase te concentras en la app.

## 3.2 SPA y PWA

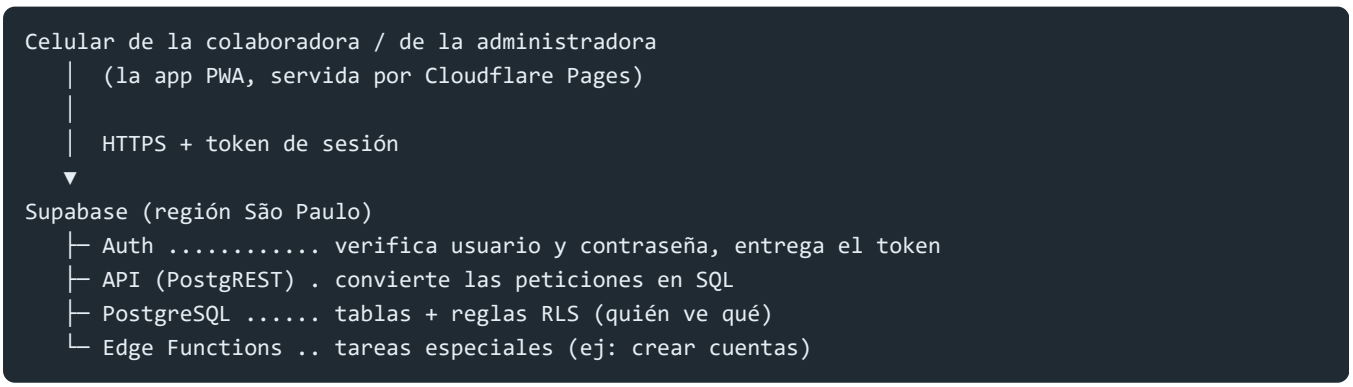
> **SPA (Single Page Application).** Aplicación de una sola página: se carga una vez y luego cambia de pantalla sin recargar, como ya hace tu prototipo con el menú ☰.

> **PWA (Progressive Web App).** Una página web que se comporta como una app: se "instala" en la pantalla de inicio con su ícono, abre a pantalla completa y **sigue funcionando sin internet**. Usa un *manifest* (ficha con nombre, ícono y colores) y un *service worker* (ver capítulo 9). Ventaja: no pasas por Play Store ni App Store y una sola versión sirve para Android y iPhone.

## 3.3 Jamstack: archivos estáticos + API

> **Sitio estático y Jamstack.** Un sitio **estático** son archivos (HTML, CSS, JavaScript) que se entregan tal cual, sin un servidor que los arme en cada visita. **Jamstack** es la arquitectura de "JavaScript + API + archivos estáticos": la app es estática y los datos vienen de una API. Es barato (hay hosting gratis), rápido y difícil de tumbar.

## 3.4 Cómo se conecta todo



### 3.5 Alternativas que descartamos

| Alternativa | Por qué no, por ahora |
|---|---|
| Servidor propio (Node/Express, Java/Spring, PHP/Laravel) | Más código y mantenimiento. Hay que pagar y cuidar un servidor. |
| Firebase (Google) | Funciona bien, pero su base de datos no es relacional (NoSQL). Para cajas, ventas y un estado de resultados conviene SQL. |
| App nativa (Android/iOS) | Dos apps, publicación en tiendas y cuentas de desarrollador pagadas. La PWA cubre lo que necesitas. |
| Servidor virtual (VPS) propio | Más barato a gran escala, pero tú haces las actualizaciones de seguridad y los respaldos. Opción para más adelante. |

# 4. Herramientas: qué es cada una y para qué la usaremos

## 4.1 Para escribir código

| Herramienta | Qué es | Para qué en este proyecto |
|---|---|---|
| **Visual Studio Code** | Editor de código gratuito de Microsoft. Ya lo usas. | Escribir el código, con extensiones para TypeScript, ESLint y Prettier. |
| **Node.js 22 LTS** | Ejecuta JavaScript en tu PC. *LTS* = *Long Term Support*, versión con soporte largo, la estable. | Correr Vite, las pruebas y la CLI de Supabase. |
| **npm** | Gestor de paquetes que viene con Node. | Instalar dependencias y correr comandos ( `npm run dev` ). |
| **TypeScript** | JavaScript con **tipos**: declaras que `precio` es un número y el editor te avisa si por error le pasas texto. | Evitar errores como `"5" + 3 = "53"` en cálculos de caja. |
| **Vite** | Herramienta de construcción (*bundler*): mientras programas, actualiza la página al instante; al terminar, junta y comprime todo en archivos optimizados (*build*). | Dividir el `index.html` actual en módulos y generar la versión de producción en la carpeta `dist/` . |

> **Módulo.** Un archivo de código con una sola responsabilidad (por ejemplo, todo lo de Caja) que exporta funciones para que otros archivos las usen. Es lo contrario de tener 1.200 líneas en un solo archivo.

> **Framework.** Un conjunto de herramientas y reglas para construir interfaces (React, Vue, Angular, Svelte). Recomendación: **no usar framework** por ahora y conservar tu código actual separado en módulos. Ya funciona y es liviano en celulares baratos. Si más adelante el proyecto crece mucho, migrar a React o Svelte es una opción.

## 4.2 Base de datos y backend

| Herramienta | Qué es | Para qué |
|---|---|---|
| **Supabase** | Plataforma BaaS de código abierto construida sobre PostgreSQL. Incluye Auth (login), API automática, Storage (archivos), Edge Functions y un panel web (Studio). | Todo el backend: datos, cuentas y permisos. |
| **PostgreSQL** | Base de datos relacional que está dentro de Supabase. | Guardar cajas, pedidos, gastos, etc. |
| **Supabase CLI** | Programa de línea de comandos de Supabase. *CLI* = *Command Line Interface*, se usa escribiendo comandos en la terminal. | Levantar Supabase en tu PC, crear migraciones, subirlas a producción, generar tipos. |
| **Docker Desktop** | Programa que ejecuta **contenedores**: "cajas" con un programa y todo lo que necesita, aisladas de tu sistema. | La CLI de Supabase lo usa para correr Postgres y el resto de Supabase en tu PC. |

| | | |
|---|---|---|
| **Supabase Studio** | Panel web para ver tablas, escribir SQL y administrar usuarios. | Revisar los datos mientras desarrollas ( `localhost:54323` ). |
| **supabase-js** | Librería de JavaScript para hablar con Supabase. | Reemplaza `load()` y `save()` de localStorage. |
| **Edge Functions** | Pequeñas funciones que corren en servidores de Supabase (escritas en TypeScript, motor *Deno*). | Crear cuentas de colaboradores de forma segura (ver capítulo 8). |

**Contenedor e imagen.** Una **imagen** es la "receta" congelada de un programa con su entorno (por ejemplo, "Postgres 15"). Un **contenedor** es esa imagen en ejecución. Así tu PC corre exactamente la misma base de datos que producción, sin instalarla a mano.

**localhost.** Nombre que significa "esta misma computadora". `localhost:54323` es el programa que escucha en el puerto 54323 de tu PC. Un **puerto** es un número que identifica qué programa atiende una conexión.

## 4.3 Calidad y pruebas

| Herramienta | Qué es | Para qué |
|---|---|---|
| **ESLint** | *Linter*: revisa el código buscando errores y malas prácticas sin ejecutarlo. | Detectar variables sin usar, comparaciones peligrosas, etc. |
| **Prettier** | Formateador: ordena espacios, comillas y saltos de línea automáticamente. | Que todo el código tenga el mismo estilo. |
| **Husky + lint-staged** | Ejecutan comandos automáticamente antes de cada *commit* (*git hook*). | No dejar guardar código que no pasa ESLint o Prettier. |
| **Vitest** | Herramienta de pruebas unitarias, hecha para Vite. | Probar cálculos: caja esperada, estado de resultados, costo por litro. |
| **Playwright** | Controla un navegador real como si fuera una persona (prueba de extremo a extremo). | Simular un celular: entrar, abrir caja, vender, cobrar, cerrar. |
| **pgTAP** | Pruebas escritas en SQL que corren dentro de Postgres ( `supabase test db` ). | Verificar que una colaboradora no puede ver la caja de otra. |

## 4.4 Publicación y operación

| Herramienta | Qué es | Para qué |
|---|---|---|
| **Git** | Control de versiones: guarda cada cambio con autor, fecha y descripción, y permite volver atrás. | Historial del proyecto. |
| **GitHub** | Sitio que guarda repositorios Git en internet y agrega revisiones, tareas y automatización. | Repositorio privado, Pull Requests, GitHub Actions. |
| **GitHub Actions** | Servicio que ejecuta tareas automáticas cuando pasa algo en el repositorio (un push, una hora fija). | Correr pruebas, desplegar la base de datos y hacer respaldos nocturnos. |

| | | |
|---|---|---|
| **Cloudflare Pages** | Hosting de sitios estáticos sobre la red mundial de Cloudflare (CDN). | Publicar la app. Cada rama tiene su propia URL de prueba. |
| **Sentry** | Servicio que recibe y agrupa los errores que ocurren en los celulares de los usuarios. | Enterarte de un error antes de que te lo cuenten. |
| **UptimeRobot** | Revisa cada pocos minutos si tu sitio responde y te avisa si se cae. | Alerta si la app o la base de datos no responden. |
| **Dependabot** | Robot de GitHub que avisa y propone actualizar dependencias con fallas de seguridad. | Mantener las librerías seguras sin revisarlas a mano. |

**CDN (Content Delivery Network).** Red de servidores repartidos por el mundo que guardan copias de tus archivos y los entregan desde el más cercano al usuario. Por eso la app carga rápido aunque el servidor "original" esté lejos.

**Hosting.** Servicio que mantiene tu sitio publicado en internet, accesible las 24 horas.

## 5. Organización del proyecto

**Estructura por funcionalidades (feature-based).** Forma de ordenar carpetas en la que cada parte del negocio (pedidos, caja, producción…) tiene su propia carpeta con todo lo suyo. Es más fácil de entender que separar "por tipo de archivo" (todas las pantallas juntas, todos los estilos juntos).

```
heladeria/
├─ src/
│  ├─ main.ts              ← arranque: sesión, menú ☰, tema
│  ├─ datos/
│  │  ├─ cliente.ts        ← conexión a Supabase
│  │  ├─ cola-offline.ts   ← lo anotado sin señal (IndexedDB)
│  │  └─ tipos.ts          ← tipos generados desde la base de datos
│  ├─ modulos/
│  │  ├─ sesion/           ← pantallas de inicio de sesión
│  │  ├─ pedidos/  caja/  produccion/  gastos/
│  │  ├─ personal/  equipos/  sobrantes/
│  │  └─ estado-resultados/
│  ├─ ui/                  ← modal, avisos, panel deslizante, PDF
│  └─ estilos/temas.css    ← Clásico, Oscuro, Noche, Fresa y menta
├─ public/                 ← íconos de la app, manifest
├─ supabase/
│  ├─ config.toml          ← configuración local de Supabase
│  ├─ migrations/          ← cambios de tablas, uno por archivo .sql
│  ├─ functions/crear-usuario/  ← Edge Function
│  ├─ seed.sql             ← datos iniciales para desarrollo
│  └─ tests/               ← pruebas pgTAP de permisos
├─ tests/                  ← Vitest (unitarias) y Playwright (e2e)
├─ .github/workflows/      ← ci.yml, deploy.yml, respaldo.yml
├─ .env.example            ← lista de variables, SIN valores secretos
├─ .gitignore              ← lo que Git no debe subir (.env, node_modules, dist)
├─ package.json
└─ README.md
```

**.env y .env.example.** El archivo `.env` tiene las variables de entorno reales de tu PC y **no se sube** a Git. `.env.example` sí se sube: lista las variables necesarias con valores de ejemplo, para que cualquiera sepa qué configurar.

**Seed (semilla).** Datos iniciales que se cargan automáticamente en desarrollo (sabores, precios, un usuario de prueba), para no escribirlos a mano cada vez que reinicias la base.

**Separación de responsabilidades.** Cada parte del código hace una sola cosa. Las pantallas no hablan directo con Supabase: llaman a funciones de `src/datos/`. Así, si mañana cambias Supabase por otra cosa, solo tocas esa carpeta.

# 6. Git y forma de trabajo

**Commit.** Una "foto" guardada de los cambios con un mensaje que explica qué se hizo. **Push** es subir tus commits a GitHub. **Pull** es bajar los cambios de GitHub a tu PC.

**Rama (branch).** Una línea de trabajo paralela. Puedes hacer cambios en una rama sin afectar la principal y, cuando estén listos, unirlos (**merge**).

## 6.1 Ramas que usaremos

| Rama | Para qué | Se despliega en |
|---|---|---|
| `main` | Lo que está en producción. Solo recibe cambios ya probados. | Producción |
| `develop` | Cambios terminados, en ensayo. | Entorno de pruebas |
| `feature/nombre` | Una mejora concreta (ej: `feature/login`). | URL de vista previa |
| `fix/nombre` | Corrección de un error. | URL de vista previa |

El camino de cada cambio: `feature/…` → Pull Request a `develop` → pruebas en celulares → Pull Request de `develop` a `main` → producción.

**Pull Request (PR).** Una solicitud en GitHub para unir una rama con otra. Muestra las diferencias, ejecuta las pruebas automáticas y permite revisar antes de aceptar. Aunque trabajes solo, úsalo: es tu último control antes de producción.

**Protección de ramas (branch protection).** Regla de GitHub que impide subir directo a `main` y exige que las pruebas pasen antes de unir un PR.

## 6.2 Mensajes de commit: Conventional Commits

**Conventional Commits.** Convención para escribir mensajes de commit con un prefijo que dice qué tipo de cambio es. Ya lo usas (`fix(ui): …`).

```
feat(caja): registrar retiros desde el panel deslizante
fix(pedidos): no duplicar pedidos al reconectar
refactor(datos): separar cliente de Supabase
test(caja): probar cálculo de dinero esperado
docs: explicar despliegue en el README
chore(deps): actualizar supabase-js
```

## 6.3 Versiones: SemVer

**Versionado semántico (SemVer).** Numerar versiones como `MAYOR.MENOR.PARCHE`, por ejemplo `1.4.2`. Subes el **parche** al corregir un error, el **menor** al agregar algo nuevo sin romper nada y el **mayor** cuando cambias algo de forma incompatible. Cada paso a producción se marca con un **tag** (etiqueta) en Git: `v1.0.0`. Así sabes exactamente qué versión está en producción y puedes volver a ella.

**CHANGELOG.** Archivo con la lista de cambios de cada versión, en lenguaje simple. Útil para contarle a tu mamá qué hay de nuevo.

**Importante:** el repositorio actual es **público** (GitHub Pages gratis lo requería). Para el proyecto real crea un repositorio **privado**: Cloudflare Pages publica desde repositorios privados sin costo.

# 7. Base de datos: diseño y buenas prácticas

## 7.1 Tablas principales

| Tabla | Columnas clave | Hoy en el prototipo es… |
|---|---|---|
| `perfiles` | id (= id de Auth), nombre, usuario, teléfono, rol (*admin* / *colaborador*), pago_diario, activo | `S.empleados` y cuenta admin |
| `puntos_venta` | id, nombre | `config.puntos` |
| `sabores`, `productos` | id, nombre, base, activo / precio, reventa, costo | `config.sabores`, `config.productos` |
| `cajas` | id, colaborador_id, punto_id, monto_inicial, abierta_en, cerrada_en, contado | `S.cajas` |
| `reemplazos` | id, caja_id, reemplazante_id, desde, hasta, entregado | reemplazos de la caja |
| `pedidos` | id, caja_id, numero, estado, metodo, pagado_en, invitado_para | `S.pedidos` |
| `pedido_items` | id, pedido_id, tipo, producto_id, precio, cantidad | vasos y productos del pedido |
| `pedido_item_sabores` | item_id, sabor_id | sabores de cada vaso |
| `gastos`, `retiros` | id, fecha, categoria, monto, caja_id (opcional) | pagos y retiros |
| `produccion`, `produccion_insumos` | lote, sabor, litros / insumo, costo | `S.produccion` |
| `sobrantes`, `activos`, `incidencias` | igual que hoy | igual que hoy |
| vista `v_estado_resultados` | ingresos, costos, mermas, utilidad por periodo | la función `calcER` |

**Clave primaria (PK) y clave foránea (FK).** La **clave primaria** identifica cada fila sin repetirse (el `id`). La **clave foránea** es una columna que apunta a la clave primaria de otra tabla: `pedidos.caja_id` apunta a `cajas.id`. La base de datos impide guardar un pedido de una caja que no existe. Esto se llama **integridad referencial**.

**Normalización.** Diseñar las tablas para que cada dato se guarde en un solo lugar. Por ejemplo, el nombre del sabor vive solo en `sabores` y los vasos guardan su `id`. Si cambias el nombre, cambia en todo el sistema. En el prototipo, las cajas guardan el nombre de la persona como texto; en la base real guardarán su `id`.

**Tabla intermedia (relación muchos a muchos).** Un vaso puede tener varios sabores y un sabor está en muchos vasos. Eso se resuelve con una tabla extra ( `pedido_item_sabores` ) que une ambos.

**Vista (view).** Una consulta SQL guardada con nombre, que se usa como si fuera una tabla. Sirve para el estado de resultados: la base de datos hace los cálculos y la app solo los muestra.

> **Índice.** Estructura que acelera las búsquedas, como el índice de un libro. Se crean en las columnas por las que más se filtra: `pedidos(caja_id)`, `gastos(fecha)`.

### 7.2 Reglas de oro para los datos

- **Dinero con `numeric(10,2)`**, nunca con decimales "flotantes". En JavaScript `0.1 + 0.2 = 0.30000000000000004`; en una caja eso termina en faltantes de centavos.
- **Fechas y horas con `timestamptz`** (con zona horaria) y la hora puesta por el servidor (`default now()`), no por el celular. La zona de Bolivia es `America/La_Paz` (UTC−4).
- **Identificadores UUID generados en el celular.** Así un pedido anotado sin señal tiene su id desde el inicio y, si se envía dos veces, no se duplica.
- **Restricciones (constraints)** en la base: `precio > 0`, `cantidad >= 1`, `estado in ('abierto','pagado','invitado')`. Es la última línea de defensa si la app tiene un error.
- **Borrado real para lo cancelado**, como decidiste: un pedido cancelado se elimina con `delete`. Para datos de dinero ya cobrado conviene, en cambio, no borrar sino anular con registro de quién y cuándo (ver "auditoría" en el glosario).

> **UUID.** Identificador de 128 bits, como `3f1c2a9e-…`, prácticamente imposible de repetir. Se puede generar en cualquier celular sin preguntar a la base de datos.

> **Idempotencia.** Que repetir una operación dé el mismo resultado que hacerla una vez. Enviar dos veces "guardar pedido con id X" deja un solo pedido X. Es clave cuando la señal es mala.

> **Transacción.** Un grupo de operaciones que se hacen todas o ninguna. Al cerrar la caja se guardan el conteo, los sobrantes y el estado; si algo falla a la mitad, no queda nada a medias.

### 7.3 Migraciones

> **Migración.** Un archivo SQL con un cambio de la estructura de la base de datos (crear una tabla, agregar una columna), con fecha en el nombre. Se guardan en Git y se aplican en orden en cada entorno. Así desarrollo, pruebas y producción tienen siempre la misma estructura y sabes qué cambió y cuándo. **Nunca** cambies tablas de producción a mano desde el panel.

```
# crear una migración nueva (genera supabase/migrations/20261102120000_crear_tablas.sql)
npx supabase migration new crear_tablas
# borrar y recrear la base local aplicando todas las migraciones + seed.sql
npx supabase db reset
# generar los tipos de TypeScript a partir de las tablas
npx supabase gen types typescript --local > src/datos/tipos.ts
```

# 8. Inicio de sesión y permisos

**Autenticación y autorización.** **Autenticación** es comprobar *quién eres* (usuario y contraseña). **Autorización** es decidir *qué puedes hacer* (la colaboradora vende; la administradora ve el estado de resultados). Son dos cosas distintas y las dos se hacen en el servidor.

## 8.1 Cómo funcionará

1. No hay registro público. En Supabase se desactiva *"Allow new users to sign up"*.
2. La administradora crea cada cuenta desde **Personal**. La app llama a una **Edge Function** `crear-usuario`, que comprueba que quien la llama es administradora y crea el usuario con la llave secreta (que solo existe en el servidor).
3. Supabase Auth trabaja con correo. Como las colaboradoras pueden no tener uno, se usa un correo interno: el usuario `erika` se guarda como `erika@heladeria.local`, con la confirmación por correo desactivada. Ella solo escribe "erika" y su contraseña.
4. Al entrar, Supabase entrega un **token JWT** que la app envía en cada petición. La base de datos lee ese token para saber quién es.
5. El enlace con `#admin` solo cambia la pantalla de inicio. Lo que realmente protege es el **rol** guardado en `perfiles` y las reglas RLS.

**Token y JWT.** Un **token** es un "pase" digital que el servidor te entrega al iniciar sesión, para no enviar la contraseña en cada petición. **JWT** (*JSON Web Token*) es un formato de token firmado: contiene datos como el id del usuario y una firma que impide falsificarlo. Caduca a la hora y se renueva solo.

**Hash de contraseña.** Transformación de una sola vía: de la contraseña se obtiene un código, pero del código no se puede volver a la contraseña. El servidor guarda solo el hash. Supabase Auth usa **bcrypt**, un algoritmo lento a propósito, para que adivinar contraseñas sea muy costoso. (El prototipo usa SHA-256 en el navegador: sirve para ordenar, no para proteger.)

## 8.2 RLS: seguridad a nivel de fila

**RLS (Row Level Security).** Función de PostgreSQL que aplica reglas (*policies*) a cada fila. Aunque alguien manipule la app o llame a la API directamente, la base de datos solo le devuelve las filas que su regla permite. Es **la** pieza de seguridad más importante del proyecto: toda tabla debe tener RLS activado.

```
-- activar RLS en la tabla
alter table pedidos enable row level security;

-- la colaboradora ve solo los pedidos de sus cajas; la administradora ve todo
create policy "ver pedidos" on pedidos for select using (
  es_admin() or exists (
    select 1 from cajas c where c.id = pedidos.caja_id and c.colaborador_id = auth.uid()
  )
);

-- solo puede crear pedidos en su caja abierta
create policy "crear pedidos" on pedidos for insert with check (
  exists (select 1 from cajas c where c.id = caja_id
          and c.colaborador_id = auth.uid() and c.cerrada_en is null)
);
```

`auth.uid()` devuelve el id del usuario del token. `es_admin()` es una función tuya que revisa el rol en `perfiles`.

## 8.3 Las llaves de Supabase

| Llave | Dónde va | Riesgo |
|---|---|---|
| **Publishable key** ( `sb_publishable_…` ; antes llamada *anon key*) | En la app. Es pública: cualquiera puede verla. | Ninguno, si RLS está bien hecho. |
| **Secret key** ( `sb_secret_…` ; antes *service_role*) | Solo en Edge Functions y en secretos de GitHub. | **Total.** Se salta RLS. Si se filtra, se cambia de inmediato. |

# 9. Funcionar sin internet

En un puesto de helados la señal móvil falla. Si la app necesita internet para anotar un pedido, la venta se frena. Por eso este capítulo no es opcional.

**Service worker.** Un archivo JavaScript que el navegador instala aparte de la página y que intercepta las peticiones. Guarda una copia de la app (HTML, CSS, JS, íconos) para que abra aunque no haya señal. Lo genera **vite-plugin-pwa** usando **Workbox**, una librería de Google para service workers.

**Caché.** Copia guardada de algo para no volver a descargarlo. El service worker guarda la app en caché.

**IndexedDB y Dexie.js.** **IndexedDB** es una base de datos que trae el navegador, con mucha más capacidad y orden que localStorage. **Dexie.js** es una librería que la hace fácil de usar.

**Offline-first y cola de sincronización.** **Offline-first** significa diseñar la app para que funcione primero sin internet y sincronice después. Cada pedido se guarda en IndexedDB en una **cola** (lista de pendientes) y se envía a Supabase cuando hay señal. Si el envío falla, se reintenta.

## 9.1 Flujo de un pedido

1. La colaboradora cobra. El pedido se guarda al instante en IndexedDB con su UUID y queda marcado "pendiente de subir".
2. Si hay señal, se envía a Supabase. Al confirmar, se marca como "subido".
3. Si no hay señal, la app muestra un aviso pequeño (por ejemplo "3 sin subir") y reintenta al recuperar conexión (evento `online`), al abrir la app y cada cierto tiempo.
4. Como el id es el mismo, reenviar no duplica (idempotencia, con `upsert`).

**Upsert.** Operación que inserta la fila si no existe y la actualiza si ya existe ("update + insert").

**Particularidades del iPhone (Safari):** no tiene "sincronización en segundo plano", así que la app sincroniza cuando está abierta. Además, Safari puede borrar los datos de sitios no usados en varios días si la app no está instalada en la pantalla de inicio. Pide a todas que la instalen con *Compartir → Agregar a inicio*.

# 10. Calidad del código y pruebas

**Prueba automatizada (test).** Un pequeño programa que usa tu código y comprueba que el resultado es el esperado. Se ejecuta en segundos, siempre igual, cada vez que cambias algo. Así un cambio en Pedidos no rompe la Caja sin que te enteres.

## 10.1 La pirámide de pruebas

| Tipo | Qué prueba | Herramienta | Cantidad |
|---|---|---|---|
| **Unitarias** | Una función sola: "con 50 de monedas, 120 en efectivo, 10 de hielo y 20 de retiro, deben haber 140". | Vitest | Muchas |
| **De integración** | Varias piezas juntas, por ejemplo la app con la base de datos local. Incluye las de permisos. | Vitest, pgTAP | Algunas |
| **De extremo a extremo (E2E)** | El sistema completo como lo usa una persona, en un navegador real. | Playwright | Pocas, las críticas |

```
// tests/caja.test.ts — ejemplo con Vitest
import { test, expect } from 'vitest'
import { dineroEsperado } from '../src/modulos/caja/calculos'

test('descuenta pagos y retiros', () => {
  expect(dineroEsperado({ inicial: 50, efectivo: 120, pagos: 10, retiros: 20 })).toBe(140)
})
```

## 10.2 Qué probar sí o sí

- Cálculos de dinero: caja esperada, faltante y sobrante, reemplazos, estado de resultados.
- Permisos (pgTAP): la colaboradora A no ve ni modifica las cajas de B, y una colaboradora no puede leer gastos ni el estado de resultados.
- Flujo E2E: iniciar sesión, abrir caja, crear pedido, cobrar, cerrar caja y descargar el PDF.
- Modo sin conexión: Playwright puede simular "sin internet" y comprobar que el pedido se sube al volver.

**Cobertura (coverage).** Porcentaje del código que ejecutan las pruebas. Es una guía, no una meta: más vale 100 % en los cálculos de dinero que 80 % repartido en botones.

**Refactorizar.** Mejorar la organización del código sin cambiar lo que hace. Las pruebas te dan la seguridad para hacerlo.

# 11. Integración y despliegue continuo

**CI (Integración Continua).** Cada vez que subes código, un servidor ejecuta automáticamente el linter, las pruebas y el build. Si algo falla, te avisa y no se puede unir el cambio.

**CD (Despliegue Continuo).** Cuando un cambio se une a la rama correspondiente, se publica automáticamente en su entorno, sin pasos a mano.

**Pipeline y workflow.** Un **pipeline** es la cadena de pasos automáticos (instalar → revisar → probar → construir → desplegar). En GitHub Actions cada pipeline se describe en un archivo **workflow** en formato **YAML** (texto con sangrías) dentro de `.github/workflows/`.

## 11.1 Los tres workflows

| Archivo | Cuándo corre | Qué hace |
|---|---|---|
| `ci.yml` | En cada push y Pull Request | ESLint, verificación de tipos, Vitest, base local con `supabase start` + `supabase test db`, Playwright y `npm run build`. |
| `deploy.yml` | Al unir a `develop` o `main` | `supabase link` al proyecto que corresponda + `supabase db push` (aplica migraciones nuevas). Cloudflare Pages publica la app por su cuenta. |
| `respaldo.yml` | Todas las noches (cron) | `supabase db dump` de producción y lo guarda cifrado (capítulo 14). |

```yaml
# .github/workflows/ci.yml (resumen)
name: CI
on: [push, pull_request]
jobs:
  probar:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22 }
      - run: npm ci
      - run: npm run lint && npm run typecheck
      - run: npx vitest run
      - run: npx supabase start && npx supabase test db
      - run: npx playwright install --with-deps && npx playwright test
      - run: npm run build
```

**Cron.** Forma de programar tareas a horas fijas. `0 7 * * *` significa "todos los días a las 07:00 UTC", que son las 03:00 en Bolivia.

**GitHub Secrets.** Lugar seguro en la configuración del repositorio para guardar secretos (token de Supabase, contraseña de la base). Los workflows los usan sin que aparezcan en el código ni en los registros.

# 12. Los tres entornos y el paso a producción

| Entorno | Base de datos | App | Quién lo usa |
|---|---|---|---|
| **Desarrollo** | Supabase local en Docker | `npm run dev` en `localhost:5173` | Tú |
| **Pruebas** (staging) | Proyecto Supabase `heladeria-pruebas` | Cloudflare Pages, rama `develop` | Tú, y quizá tu mamá y una colaboradora para ensayar |
| **Producción** | Proyecto Supabase `heladeria-prod` (región São Paulo) | Cloudflare Pages, rama `main` | La heladería |

> **Staging.** Entorno de ensayo que se parece lo más posible a producción, pero con datos de prueba. Lo que funciona aquí tiene muy alta probabilidad de funcionar en producción.

> **Región.** Ubicación física de los servidores. São Paulo ( `sa-east-1` ) es la más cercana a Bolivia; menos distancia significa respuestas más rápidas (menos **latencia**).

## 12.1 Paso a paso

### Fase 0 — Preparar tu PC

```
winget install OpenJS.NodeJS.LTS
winget install Docker.DockerDesktop
npm create vite@latest heladeria -- --template vanilla-ts
cd heladeria && npm i @supabase/supabase-js dexie && npm i -D supabase vite-plugin-pwa vitest
@playwright/test eslint prettier
```

### Fase 1 — Base de datos local

```
npx supabase init          # crea la carpeta supabase/
npx supabase start         # levanta todo en Docker
#   API ........ http://localhost:54321
#   Postgres ... localhost:54322
#   Studio ..... http://localhost:54323
npx supabase migration new crear_tablas
npx supabase db reset      # aplica migraciones + seed
```

Después: mover el código del prototipo a módulos, cambiar `load()` / `save()` por `src/datos/` , escribir RLS y pruebas.

### Fase 2 — Entorno de pruebas

1. Crear el proyecto `heladeria-pruebas` en supabase.com.
2. En Cloudflare: *Workers & Pages → Create → Pages → Connect to Git*, elegir el repositorio. Build command `npm run build` , carpeta de salida `dist` .
3. Configurar las variables `VITE_SUPABASE_URL` y `VITE_SUPABASE_PUBLISHABLE_KEY` para la rama de vista previa (Preview).
4. Ensayar con celulares reales unos días, incluso en paralelo con el prototipo actual.

**Fase 3 — Producción**

1. Crear `heladeria-prod` en São Paulo. Guardar la contraseña de la base en un gestor de contraseñas.
2. Cargar en GitHub Secrets: `SUPABASE_ACCESS_TOKEN`, `SUPABASE_DB_PASSWORD`, `SUPABASE_PROJECT_ID_PROD`.
3. En Supabase Auth: desactivar el registro público y la confirmación por correo.
4. Pull Request `develop → main`, esperar que el CI pase y unirlo. Las migraciones se aplican y la app se publica sola.
5. Crear la cuenta de la administradora y, desde la app, las de las colaboradoras.
6. Pasar los datos del prototipo: descargar el **respaldo JSON** de cada celular y cargarlo con un script (`scripts/importar-respaldo.ts`).
7. En cada celular: abrir la URL y *Agregar a la pantalla de inicio*.
8. Marcar la versión: `git tag v1.0.0 && git push --tags`.

> **Rollback (vuelta atrás).** Regresar a la versión anterior si la nueva falla. En Cloudflare Pages es un clic (*Deployments → Rollback*). En la base de datos no hay un clic: por eso las migraciones se prueban antes en staging y se escriben de forma que no rompan la versión anterior.

# 13. Seguridad

**OWASP Top 10.** Lista de los 10 riesgos de seguridad web más comunes, publicada por la fundación OWASP. Vale la pena leerla una vez; los puntos de abajo cubren los que aplican a este proyecto.

| Riesgo | Qué es | Cómo lo cubrimos |
|---|---|---|
| **Control de acceso roto** | Alguien ve o cambia datos que no le corresponden. | RLS en todas las tablas + pruebas pgTAP. |
| **XSS** (*Cross-Site Scripting*) | Alguien escribe código en un campo (por ejemplo, en "para quién es") y ese código se ejecuta en el celular de otra persona. | Escapar todo texto antes de mostrarlo (la función `esc()` que ya existe) y una política CSP. |
| **Secretos expuestos** | Contraseñas o llaves subidas a Git. | `.gitignore` con `.env`, GitHub Secrets y la función de escaneo de secretos de GitHub (*secret scanning*). |
| **Contraseñas débiles / fuerza bruta** | Probar miles de contraseñas hasta acertar. | Supabase Auth limita intentos (*rate limiting*). Exigir 8 o más caracteres. |
| **Dependencias vulnerables** | Una librería con una falla conocida. | Dependabot + `npm audit` en el CI. |
| **Tráfico interceptado** | Leer datos en una red Wi-Fi pública. | HTTPS obligatorio (Cloudflare y Supabase lo dan por defecto). |

**CSP (Content Security Policy).** Cabecera HTTP que le dice al navegador de dónde puede cargar scripts y a dónde puede conectarse. Si alguien logra inyectar código, el navegador lo bloquea. En Cloudflare Pages se define en un archivo `public/_headers`.

**Principio de mínimo privilegio.** Cada persona y cada programa tiene solo los permisos que necesita. La colaboradora no ve el estado de resultados; la app no usa la llave secreta.

**Rate limiting.** Limitar cuántas peticiones puede hacer alguien en un tiempo. Frena ataques de fuerza bruta.

**2FA (verificación en dos pasos).** Además de la contraseña, un código del celular. Supabase lo ofrece como MFA. Conviene activarlo en tus cuentas de GitHub, Supabase y Cloudflare: si alguien entra ahí, tiene todo.

## 13.1 Datos personales

El sistema no guarda datos de clientes. Sí guarda nombres y teléfonos de colaboradoras: guarda solo lo necesario, que solo la administradora pueda verlo y bórralos cuando alguien deje de trabajar. Si en el futuro necesitas **facturación electrónica** del SIN de Bolivia, es un proyecto aparte con requisitos propios; esta guía no lo cubre.

# 14. Operación: respaldos y monitoreo

## 14.1 Respaldos

**Respaldo (backup) y restauración (restore).** Un **respaldo** es una copia de la base de datos guardada en otro lugar. **Restaurar** es volver a cargarla. Un respaldo que nunca restauraste no sabes si sirve.

**RPO y RTO.** **RPO** (*Recovery Point Objective*): cuántos datos aceptas perder; con respaldo diario, como máximo un día. **RTO** (*Recovery Time Objective*): cuánto tiempo aceptas estar sin sistema mientras restauras. Defínelos con tu mamá: por ejemplo, RPO de 1 día y RTO de 2 horas.

**Dump.** Archivo de texto con todo el contenido de la base en forma de SQL. `supabase db dump` lo genera.

- **Plan Free:** Supabase **no hace respaldos que puedas descargar**. Por eso existe `respaldo.yml`: cada noche hace el *dump* (estructura y datos), lo comprime, lo cifra con una contraseña y lo guarda en un repositorio privado aparte o en Google Drive.
- **Plan Pro:** respaldos diarios automáticos con 7 días de historial. La restauración a un minuto exacto (**PITR**, *Point-in-Time Recovery*) es un complemento pagado aparte.
- **Simulacro mensual:** descargar el último respaldo, cargarlo en tu base local ( `supabase db reset` + cargar el dump) y comprobar que las ventas están. Anota la fecha en el README.

## 14.2 Monitoreo

**Monitoreo y observabilidad.** **Monitorear** es vigilar que el sistema funcione y recibir alertas. La **observabilidad** es tener suficiente información (errores, registros, métricas) para entender por qué algo falló.

**Log (registro).** Línea de texto que un programa escribe cuando pasa algo ("usuario erika inició sesión", "error al guardar pedido"). Supabase guarda logs de la API, Auth y la base en su panel.

| Qué vigilar | Con qué | Frecuencia |
|---|---|---|
| Errores en los celulares | Sentry (SDK en la app) | Alerta por correo al ocurrir |
| Que la app y la API respondan | UptimeRobot | Cada 5 minutos |
| Espacio usado en la base | Panel de Supabase → *Usage* | Una vez al mes |
| Que el respaldo nocturno funcionó | GitHub Actions avisa por correo si falla | Diario, automático |
| Dependencias con fallas | Dependabot | Semanal, automático |

## 14.3 La pausa del plan gratis

Un proyecto Free de Supabase se **pausa si no recibe actividad durante 7 días seguidos**. Los datos no se pierden; se reactiva desde el panel con un clic. Si la heladería cierra una semana (vacaciones, lluvias), reactívala antes de volver a abrir o, mejor, pasa a Pro cuando el negocio lo permita.

# 15. Costos y límites de los planes gratis

> **Verifica antes de decidir.** Estos números son los que conozco a la fecha de esta guía y cambian con frecuencia. Revisa `supabase.com/pricing`, `pages.cloudflare.com`, `github.com/pricing` y `sentry.io/pricing`.

## 15.1 Supabase

| Concepto | Free (US$ 0) | Pro (≈ US$ 25/mes por organización) |
|---|---|---|
| Tamaño de la base | 500 MB | 8 GB incluidos |
| Archivos (Storage) | 1 GB | 100 GB |
| Tráfico de salida | 5 GB/mes | 250 GB/mes |
| Usuarios activos | 50.000/mes | 100.000/mes |
| Proyectos | 2 activos | Según cómputo (crédito de US$ 10 incluido) |
| Pausa por inactividad | Sí, a los 7 días | No |
| Respaldos | No | Diarios, 7 días |
| Edge Functions | 500.000 llamadas/mes | 2 millones/mes |

**Cuánto ocuparás:** con unos 150 vasos al día salen unas 100.000 filas por año. A cerca de 1 KB por fila con índices, son **unos 100 MB al año**. El plan Free alcanza para 3 o 4 años de datos. El tráfico se mantiene bajo si la app pide solo lo del día o del periodo consultado, nunca todo el historial.

## 15.2 Lo demás

| Servicio | Plan gratis | Observación |
|---|---|---|
| Cloudflare Pages | Ancho de banda ilimitado, 500 despliegues al mes, permite uso comercial | Vercel Hobby, en cambio, **prohíbe el uso comercial**. |
| GitHub | Repositorios privados ilimitados, 2.000 minutos de Actions al mes | El CI de este proyecto usa pocos minutos. |
| Sentry | Plan Developer: 1 usuario, cupo mensual de errores | Suficiente para empezar. |
| UptimeRobot | 50 monitores cada 5 minutos | — |
| Docker Desktop | Gratis para negocios pequeños (menos de 250 empleados y de US$ 10 millones de ingresos) | — |
| Dominio propio | No hay gratis; `heladeria.pages.dev` sí lo es | Un `.com` cuesta ≈ US$ 10–11/año en Cloudflare Registrar. |

> **Dominio y DNS.** Un **dominio** es el nombre de tu sitio (`miheladeria.com`). El **DNS** es la "guía telefónica" de internet que traduce ese nombre a la dirección del servidor. Al comprar un dominio, en Cloudflare se conecta a Pages en unos clics.

## 15.3 Por etapa

| Etapa | Costo mensual |
|---|---|
| Desarrollo y ensayo | Bs 0 |
| Producción inicial (años 1 a 3) | Bs 0 (dominio opcional ≈ US$ 11/año) |
| Cuando crezca o quieras respaldos automáticos y sin pausa | ≈ US$ 25/mes (Supabase Pro) |
| Alternativa más barata a Pro | Un VPS (por ejemplo Hetzner, ≈ € 4–5/mes) con Postgres propio. Más barato, pero tú haces todo el mantenimiento. |

Los pagos en dólares se hacen con tarjeta internacional. Consulta a tu banco el tipo de cambio y los recargos.

## 16. Lista de verificación antes de producción

### Código y repositorio

- ☐ Repositorio privado, con protección de la rama `main`.
- ☐ `.env` en `.gitignore`; ningún secreto en el código ni en el historial.
- ☐ ESLint, Prettier y TypeScript sin errores.
- ☐ README con cómo instalar, correr, probar y desplegar.

### Base de datos

- ☐ Todas las tablas creadas por migraciones guardadas en Git.
- ☐ RLS activado en **todas** las tablas, con pruebas pgTAP que pasan.
- ☐ Dinero en `numeric`, fechas en `timestamptz`, ids UUID.
- ☐ Restricciones (`check`, `not null`, claves foráneas) e índices en columnas de búsqueda.

### Login

- ☐ Registro público desactivado; cuentas creadas solo por la administradora (Edge Function).
- ☐ La llave secreta solo en Edge Functions y GitHub Secrets.
- ☐ 2FA activado en GitHub, Supabase y Cloudflare.

### App

- ☐ Funciona sin internet y sincroniza al volver la señal, sin duplicar.
- ☐ Instalable como PWA (manifest, íconos, service worker).
- ☐ Probada en un Android barato y en un iPhone, en pantallas de 360 px.
- ☐ Cabeceras de seguridad (CSP) en `_headers`.

### Operación

- ☐ Respaldo nocturno funcionando y **un simulacro de restauración hecho**.
- ☐ Sentry y UptimeRobot configurados con tu correo.
- ☐ Plan de vuelta atrás conocido (rollback en Cloudflare).
- ☐ Datos del prototipo importados y revisados con tu mamá.
- ☐ Versión `v1.0.0` etiquetada y CHANGELOG iniciado.

## 17. Plan de estudio sugerido

En orden. Cada bloque tiene qué investigar y una práctica concreta sobre tu proyecto. Las fuentes son las documentaciones oficiales: son gratuitas y las más actualizadas.

| # | Tema | Qué investigar | Práctica |
|---|---|---|---|
| 1 | Git y GitHub | Ramas, merge, Pull Requests, Conventional Commits, protección de ramas. *docs.github.com* | Crear el repo privado, la rama `develop` y un PR de prueba. |
| 2 | TypeScript básico | Tipos, interfaces, módulos `import/export`. *typescriptlang.org/docs* (Handbook) | Pasar los cálculos de caja a `calculos.ts` con tipos. |
| 3 | Vite | `dev`, `build`, variables `import.meta.env`. *vite.dev/guide* | Dividir `index.html` en módulos y que funcione igual. |
| 4 | SQL y PostgreSQL | `create table`, `select` / `join` / `group by`, claves, `numeric`, `timestamptz`. *postgresql.org/docs* (Tutorial) | Escribir la migración de las tablas y la vista del estado de resultados. |
| 5 | Docker básico | Imagen, contenedor, Docker Desktop. *docs.docker.com/get-started* | `supabase start` y explorar Studio. |
| 6 | Supabase | CLI y desarrollo local, migraciones, supabase-js, Auth, RLS, Edge Functions. *supabase.com/docs* | Login con usuario interno, RLS de pedidos y la función `crear-usuario`. |
| 7 | Pruebas | Vitest, Playwright, pgTAP. *vitest.dev*, *playwright.dev*, *supabase.com/docs/guides/database/testing* | 5 pruebas de cálculos, 3 de permisos y 1 E2E del cobro. |
| 8 | PWA y offline | Manifest, service worker, vite-plugin-pwa, IndexedDB con Dexie. *vite-pwa-org.netlify.app*, *dexie.org*, *web.dev/learn/pwa* | Cola de pedidos sin señal, probada con el modo avión. |
| 9 | GitHub Actions | Workflows, jobs, secrets, cron. *docs.github.com/actions* | `ci.yml`, luego `deploy.yml` y `respaldo.yml`. |
| 10 | Cloudflare Pages | Conectar Git, builds, variables, previews, `_headers`, rollback. *developers.cloudflare.com/pages* | Publicar `develop` como entorno de pruebas. |
| 11 | Seguridad web | OWASP Top 10, XSS, CSP, manejo de secretos. *owasp.org/Top10*, *developer.mozilla.org* (sección Security) | Revisar cada `innerHTML` y escribir el `_headers`. |
| 12 | Operación | `supabase db dump`, restauración, Sentry, UptimeRobot. | Primer simulacro de restauración. |

**Consejo:** no intentes aprender todo antes de empezar. Avanza en el orden de la tabla, haciendo la práctica de cada bloque sobre tu propio proyecto. Al terminar el bloque 6 ya tendrás un sistema compartido funcionando en tu PC.

## 18. Glosario de la A a la Z

| Término | Significado |
|---|---|
| API | "Ventanilla" por la que un programa pide datos u operaciones a otro. |
| Auditoría (registro de) | Tabla que anota quién cambió qué y cuándo. Útil para operaciones de dinero. |
| Autenticación | Comprobar quién eres (usuario y contraseña). |
| Autorización | Decidir qué puedes hacer según tu rol. |
| BaaS | Backend como servicio: base de datos, login y permisos ya hechos (Supabase). |
| Backend / Frontend | Lo que corre en el servidor / lo que corre y se ve en el celular. |
| Backup / Restore | Copia de seguridad / volver a cargarla. |
| bcrypt | Algoritmo lento y seguro para guardar hashes de contraseñas. |
| Branch / Merge | Rama de trabajo paralela / unir una rama con otra. |
| Build | Versión optimizada de la app lista para publicar (carpeta `dist`). |
| Bundler | Herramienta que junta y comprime el código (Vite). |
| Caché | Copia guardada para no descargar de nuevo. |
| CDN | Red de servidores por el mundo que entrega tus archivos desde el más cercano. |
| CI / CD | Integración continua (pruebas automáticas) / despliegue continuo (publicación automática). |
| CLI | Programa que se usa escribiendo comandos en la terminal. |
| Commit / Push / Pull | Guardar cambios en Git / subirlos a GitHub / bajarlos. |
| Constraint | Regla en la base de datos que impide datos inválidos. |
| Contenedor / Imagen | Programa aislado en ejecución / su receta congelada (Docker). |
| Cron | Programación de tareas a horas fijas. |
| CSP | Cabecera que limita qué scripts puede ejecutar el navegador. |
| Dependencia | Librería de terceros que usa tu proyecto. |
| Deploy | Publicar una versión en un entorno. |
| DNS / Dominio | Traductor de nombres a direcciones / el nombre de tu sitio. |
| Dump | Archivo con todo el contenido de la base en SQL. |
| E2E | Prueba de extremo a extremo, en un navegador real. |
| Edge Function | Función pequeña que corre en los servidores de Supabase. |
| Entorno | Copia separada del sistema: desarrollo, pruebas o producción. |
| Feature-based | Ordenar carpetas por funcionalidad del negocio. |
| FK / PK | Clave foránea (apunta a otra tabla) / clave primaria (identifica la fila). |

| | |
|---|---|
| Framework | Conjunto de herramientas para construir interfaces (React, Vue…). |
| Hash | Código de una sola vía obtenido de un dato, por ejemplo de una contraseña. |
| Hosting | Servicio que mantiene tu sitio publicado. |
| HTTPS | Comunicación web cifrada. |
| Idempotencia | Repetir una operación da el mismo resultado que hacerla una vez. |
| Índice | Estructura que acelera las búsquedas en una tabla. |
| IndexedDB | Base de datos dentro del navegador. |
| JSON | Formato de texto para intercambiar datos. |
| JWT / Token | Pase digital firmado que demuestra que iniciaste sesión. |
| Latencia | Tiempo que tarda una petición en ir y volver. |
| Linter | Revisor automático de errores y malas prácticas en el código. |
| localhost / Puerto | Tu propia PC / número que identifica qué programa atiende. |
| Log | Registro de texto de lo que hace un programa. |
| LTS | Versión con soporte de largo plazo, la estable. |
| Manifest | Ficha de la PWA con nombre, ícono y colores. |
| MFA / 2FA | Verificación en dos pasos. |
| Migración | Archivo SQL versionado con un cambio en la estructura de la base. |
| Mínimo privilegio | Dar solo los permisos necesarios. |
| Módulo | Archivo de código con una sola responsabilidad. |
| Normalización | Diseñar tablas para que cada dato viva en un solo lugar. |
| npm / Node.js | Gestor de paquetes / motor para ejecutar JavaScript en tu PC. |
| numeric | Tipo de dato exacto para dinero en PostgreSQL. |
| Offline-first | Diseñar para funcionar sin internet y sincronizar después. |
| OWASP Top 10 | Lista de los riesgos de seguridad web más comunes. |
| Pipeline / Workflow | Cadena de pasos automáticos / su archivo en GitHub Actions. |
| PITR | Restaurar la base a un momento exacto (complemento pagado). |
| Policy | Regla RLS que decide qué filas ve o cambia cada usuario. |
| PostgreSQL | Base de datos relacional de código abierto. |
| Pull Request | Solicitud para unir una rama, con revisión y pruebas. |
| PWA | Web que se instala y funciona como app, incluso sin internet. |
| Rate limiting | Límite de peticiones por tiempo. |
| Refactorizar | Reordenar el código sin cambiar lo que hace. |

| | |
|---|---|
| Región | Ubicación física de los servidores. |
| Repositorio | Carpeta del proyecto con su historial en Git. |
| RLS | Seguridad a nivel de fila en PostgreSQL. |
| Rollback | Volver a la versión anterior. |
| RPO / RTO | Datos que aceptas perder / tiempo que aceptas estar sin sistema. |
| Secreto | Valor de configuración que nadie debe ver (contraseñas, llaves). |
| Seed | Datos iniciales cargados automáticamente en desarrollo. |
| SemVer / Tag | Numeración MAYOR.MENOR.PARCHE / etiqueta de versión en Git. |
| Service worker | Script del navegador que permite usar la app sin internet. |
| SPA | Aplicación de una sola página. |
| SQL | Lenguaje para crear y consultar bases de datos relacionales. |
| Staging | Entorno de ensayo parecido a producción. |
| timestamptz | Fecha y hora con zona horaria en PostgreSQL. |
| Transacción | Grupo de operaciones que se hacen todas o ninguna. |
| TypeScript | JavaScript con tipos. |
| Upsert | Insertar o actualizar según si la fila ya existe. |
| UUID | Identificador único que se puede generar en cualquier dispositivo. |
| Variable de entorno | Configuración que se entrega al programa desde afuera del código. |
| Vista (view) | Consulta SQL guardada que se usa como tabla. |
| VPS | Servidor virtual alquilado que tú administras. |
| XSS | Ataque que inyecta código en una página para que se ejecute en otro navegador. |
| YAML | Formato de texto con sangrías, usado en los workflows. |